Scheduling Algorithms

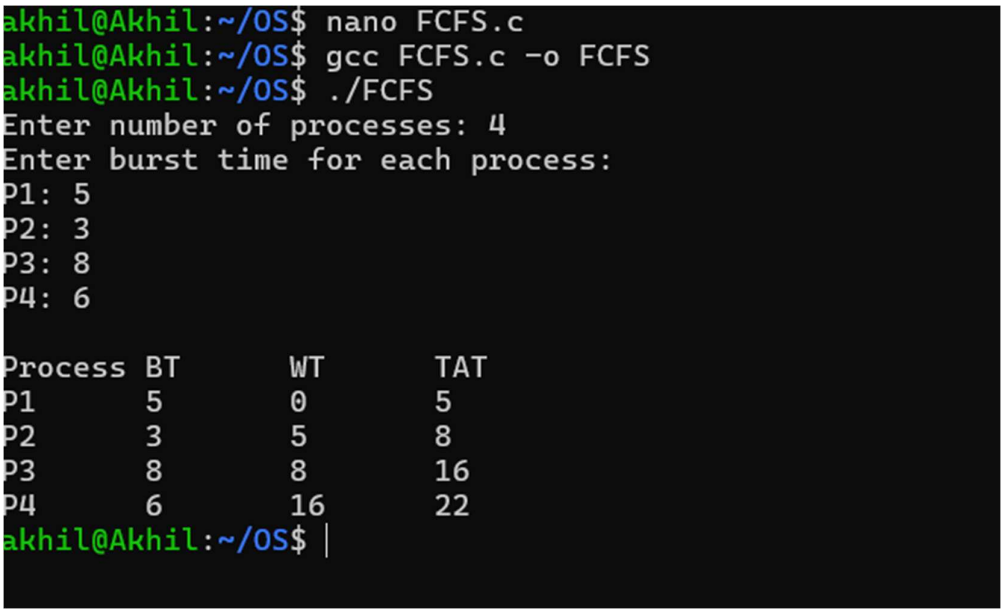
K.SAI AKHIL

2520030021

1. FCFS Scheduling algorithm

```c
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

OUTPUT



1. SJF_NONpreemptive

```c
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

OUTPUT

```
akhil@Akhil:~/OS$ gcc sfj_np.c -o sjf
akhil@Akhil:~/OS$ ./sjf
Enter number of processes: 5
Enter burst time:
P1: 7
P2: 3
P3: 10
P4: 2
P5: 5

Process BT      WT      TAT
P1      2       0       2
P2      3       2       5
P3      5       5       10
P4      7       10      17
P5      10      17      27
```

2. Priority

```
 GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {

            if (priority[i] > priority[j]) {

                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t\t%d\t%d\n",
               i + 1, bt[i], priority[i], wt[i], tat[i]);
    }

    return 0;
}
```

OUTPUT

```
akhil@Akhil:~/OS$ nano priority.c
akhil@Akhil:~/OS$ gcc priority.c -o ./priority
akhil@Akhil:~/OS$ ./priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 7
P1 Priority: 3
P2 Burst Time: 4
P2 Priority: 1
P3 Burst Time: 6
P3 Priority: 4
P4 Burst Time: 2
P4 Priority: 2
P5 Burst Time: 5
P5 Priority: 5

Process BT      Priority        WT      TAT
P1      4       1               0       4
P2      2       2               4       6
P3      7       3               6       13
P4      6       4               13      19
P5      5       5               19      24
```

3. ROUND ROBIN

```
 GNU nano 8.7.1
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin
    while (1) {

        done = 1;

        for (i = 0; i < n; i++) {

            if (rem_bt[i] > 0) {

                done = 0;

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                }
                else {
                    time = time + rem_bt[i];

                    wt[i] = time - bt[i];

                    rem_bt[i] = 0;
                }
            }
        }

        if (done == 1)
            break;
```

```
    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

return 0;
```

OUTPUT

```
akhil@Akhil:~/OS$ gcc roundRobin.c -o roundRobin
akhil@Akhil:~/OS$ ./roundRobin
Enter number of processes: 4
Enter burst time:
P1: 5
P2: 4
P3: 2
P4: 7
Enter time quantum: 2

Process BT      WT      TAT
P1      5       10      15
P2      4       8       12
P3      2       4       6
P4      7       11      18
```